Today 6:00 PM



PDF 8_SE_Lab1_SE_Prob... PDF

PDF Digital_Campus_Li... PDF

PDF Digital_Campus_Li... PDF

PDF Requirement_table... PDF

https://github.com/NIKHCODE/MyProject_Campus-Library-Reservation

I can review your **Digital Campus Library Reservation Gateway** GitHub repository against the architecture diagram and the Lab 1 deliverables you have shared.

Your architecture diagram represents a three-layer system:

- **Presentation layer:** Web/mobile interface for Student Members and the Head Librarian.
- **Application layer:** API Gateway, Catalog Search, Reservation/Hold Queue, Notification, Loan & Return, and Fine Management services.
- **Data layer:** Catalog Database, Library Database, and Queue Store, with an external Email/SMS notification service.

The diagram is broadly aligned with your SRS, particularly its architecture summary on page 2 and its five functional requirements and two non-functional requirements on pages 1–2. The assignment also explicitly requires a UML use-case diagram with at least one `«include»` and one `«extend»` relationship, plus a one-page use-case flow specification. Digital_Campus_Library_SRS.pdf

Repository review

I checked the public GitHub page: MyProject_Campus-Library-Reservation. It currently lists the README, requirements table, UML PDF, architecture image, and Jira Kanban/Scrum PDFs. GitHub

One important issue: **the README references deliverables that are not visible in the repository's root file listing**, including the problem statement, SRS, and work breakdown document. It also references an architecture subfolder, while the architecture image appears at the repository root. GitHub

What would you like me to do with the repository?

Audit the entire repository against the Lab 1 submission requirements

Fix and improve the README and folder structure

Evaluate the architecture diagram for correctness and consistency with the SRS

Give me a final submission checklist and identify missing deliverables

Review scope

All submitted documents and GitHub files

GitHub repository only

Start review

Lab_4_VibeCoding...
PDF

those are above sources and i want you to help me finish this lab assignment lab4

Understood. We'll use your existing project and assignment materials as context, but **Lab 4 is a separate VibeCoding assignment**. I'll help you complete it step by step, including debugging the Python game/puzzle, adding the required features, recording the before/after videos, and pushing the finished code to your own GitHub repository.

The Lab 4 handout specifies a 60-minute individual assignment. Its deliverables are before-and-after videos, updated code, and exported ChatGPT/tool conversation history in PDF or DOC format.

Lab_4_VibeCoding_Student_handout.pdf

Step 1 — Identify your assigned repository

Send me the GitHub URL of the repository assigned to you for Lab 4.

It should be the repository containing the broken Python game or puzzle and its `README.md`.

Once you send it, we'll proceed in this order:

1. Clone and inspect

Read the README, identify the broken code, and list every required feature.

2. Record the baseline

Run the original game and record the required 10-second video before making changes.

3. Fix and enhance

Use focused VibeCoding prompts to repair the bugs and implement each README feature. Test the changes.

4. Verify and record

Run the completed game, verify the features, and record the after-video.

5. Submit

Push the updated code to your personal repository and prepare the exported chat history and both videos under your Lab-4 submission folder. Do not raise a PR to the original `SETAPESU26` repository.

Send the **assigned repository link first**. I'll inspect its README and tailor the exact prompts and commands to that project instead of guessing what features it needs.

https://github.com/SETAPESU26/08_helicopter

Great. I've inspected the assigned repository and its README. We now know exactly what your Lab 4 requires, so we can work directly against the original specification rather than adding unnecessary features. SETAPESU26/08_...

Step 2 — Understand the four required tasks

Your assigned project is a Python/Pygame helicopter game.

Task	Required change
1. Movement and boundaries	Cap vertical speed, make direction changes responsive, and prevent the helicopter from leaving the screen at either edge.
2. Collision and game over	Detect collisions with the top and bottom obstacle walls; show a game-over message. Passing through the gap must remain safe.
3. Distance scoring	Increase the distance score during gameplay, show the final score, and reset it when a new game starts.
4. Shield mechanic	Allow a shield to absorb exactly one collision, visibly indicate when active, and deactivate it after absorbing a hit.

The README specifies Python 3.10 or later, installation using `pip install -r requirements.txt`, and launching with `python main.py`. SETAPESU26/08_...

Step 3 — Start with the original game

We need to capture the broken behavior **before modifying any code**, as required by your lab handout.

3.1 Open the repository

Use the original SETAPESU26 repository.

Open 08_helicopter on GitHub

3.2 Fork the repository

Sign in to GitHub, click **Fork**, and create your own copy. Do all your changes in your personal repository, not the original repository.

3.3 Clone your fork

Open Command Prompt or the VS Code terminal and run the following, replacing `YOUR_USERNAME` with your GitHub username.

Bash

```
git clone https://github.com/YOUR_USERNAME/08_helicopter.git
cd 08_helicopter
```

Install the dependencies and launch the original game:

Bash

```
python -m pip install -r helicopter/requirements.txt
python helicopter/main.py
```

These commands assume the repository's Python project is inside the `helicopter/` subfolder shown in the README. If your cloned folder structure differs, we'll adjust the commands accordingly.

Step 4 — Record the baseline video

Before using any AI coding tool, run the original game and record about 10 seconds of gameplay.

Try these actions:

- Hold **Up Arrow** briefly, then immediately hold **Down Arrow** to demonstrate the delayed reversal.
- Hold **Down Arrow** to demonstrate the missing bottom-boundary protection, if reproducible.

Use Windows Snipping Tool (`Win + Shift + R` , if screen recording is available) or OBS. Save the clip as `before_changes.mp4` .

Do not fix anything until you've saved this recording.

Your next action

Complete the fork and clone steps, then tell me what happens when you run the game. If you get an error, send a screenshot of the terminal. I'll guide you through the next step, then we'll use a small number of targeted prompts to implement and test all four tasks.



Perfect. Your fork is created successfully. The screenshot confirms that `NIKHCODE/SE_Lab04_08_helicopter` is your personal repository, forked from `SETAPESU26/08_helicopter` , on the `main` branch.

Next step: inspect the project files

You do not need to fork again.

1. Click the `helicopter` folder in your repository.
2. Check that it contains `main.py` and `requirements.txt` .
3. Return to the repository root and open `README.md` . Scroll down to read the complete list of tasks and features.

Before editing any code, we must record the original game for 10 seconds.

Choose how you want to proceed

VS Code on Windows — clone, run, and edit locally

GitHub website only — inspect files and edit online

Continue

My recommendation: use VS Code on Windows. It will make running the game, testing the fixes, recording before/after behavior, and committing the completed code much easier.



i did not run these commands yet

No problem. We haven't changed anything yet. Let's do it one step at a time on your Windows computer.

Step 1: Open VS Code

1. Open **Visual Studio Code**.
2. Click **Terminal** → **New Terminal** in the top menu.
3. A terminal should open at the bottom of the window.

Step 2: Clone your own repository

Your GitHub username is `NIKHCODE`, and your repository is `SE_Lab04_08_helicopter`. So, use **your fork's URL**, not the original repository URL.

Copy and paste this command into the VS Code terminal:

```bash
git clone https://github.com/NIKHCODE/SE_Lab04_08_helicopter.git
```

Press **Enter** and wait for it to finish.

Then run:

```bash
cd SE_Lab04_08_helicopter
```

Step 3: Confirm it worked

Run:

```bash
dir
```

You should see items including:

- `helicopter`
- `README.md`
- `.gitignore`

Stop here for now. Send me a screenshot of the terminal output after running these commands.

If you see an error such as `git is not recognized`, don't worry—send me the screenshot, and we'll resolve it before proceeding.

i want to do in cmd

Sure. We'll do everything using **Windows Command Prompt (CMD)**, not VS Code's terminal.

Step 1 — Open CMD

1. Press `Win + R`.
2. Type `cmd`.
3. Press **Enter**.

A black Command Prompt window will open.

Step 2 — Check whether Git is installed

Copy and paste this command into CMD:

```cmd
git --version
```

Press Enter.

- If it displays something like `git version 2.x.x`, proceed to Step 3.
- If it says `'git' is not recognized`, send me a screenshot. We'll install Git first.

Step 3 — Clone your GitHub repository

Once Git is working, run:

```cmd
git clone https://github.com/NIKHCODE/SE_Lab04_08_helicopter.git
```

Wait until the download finishes. Then enter the project folder:

cmd

```
cd SE_Lab04_08_helicopter
```

Check the files:

cmd

```
dir
```

You should see the `helicopter` folder and `README.md`.

Do Step 2 first and tell me what CMD displays. We'll proceed based on the actual output, one step at a time.



Perfect! Your repository has been cloned successfully. ✅ You're in the correct directory:

```
C:\Users\Nikhilsai\SE_Lab04_08_helicopter
```

Let's continue using CMD only.

Step 4 — Inspect the project files

First, enter the `helicopter` subfolder:

cmd

```
cd helicopter
```

Then list its contents:

cmd

```
dir
```

You should see the Python game files, including `main.py` and `requirements.txt`.

Step 5 — Read the project instructions

Go back to the repository root:

```cmd
cd ..
```

Display the README:

```cmd
type README.md
```

Read the listed tasks and features. These determine exactly what we need to fix and implement.

Step 6 — Before making changes

Do not edit the code yet. We need to run the original game and record the required 10-second baseline video first.

Run these commands now:

```cmd
cd helicopter
dir
```

Send me a screenshot of the output. I'll confirm the actual file names and give you the correct commands to install dependencies and launch the original game.



Good. We have confirmed the actual project structure. Your `helicopter` folder contains:

- `main.py` — game entry point

- `requirements.txt` — Python dependencies
- `game` — folder containing additional game code

The README also confirms that the submission requires two 10-second videos and the complete chat history link.

Step 5 — Install dependencies

You're already in the correct directory:

```
C:\Users\Nikhilsai\SE_Lab04_08_helicopter\helicopter>
```

Run this command:

```cmd
python -m pip install -r requirements.txt
```

Wait until installation finishes. If it reports an error, send me the output.

Step 6 — Run the original game

After the dependencies install successfully, run:

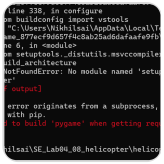
```cmd
python main.py
```

A game window should open.

1. Play for approximately 10 seconds.
2. Test the helicopter's movement and try to reproduce the broken behavior.
3. Record your screen using Windows Snipping Tool or OBS and save the video as `before_changes.mp4`.
4. Close the game after recording.

Important: Do not modify `main.py` or any other file yet. We need the original recording before any changes.

Once you've run `python main.py`, tell me whether the game opens successfully or share a screenshot of the error.



The error is clear: **Pygame failed to install because its build process could not find a required `setuptools` module.** This can often be fixed by upgrading the Python packaging tools and retrying the installation.

Let's fix it using CMD without changing your project code.

Step 1 — Upgrade the build tools

You're already in the correct `helicopter` directory. Run these commands one at a time:

cmd

```
python -m pip install --upgrade pip setuptools wheel
```

Wait for it to finish. Then run:

cmd

```
python -m pip install --upgrade setuptools
```

Step 2 — Retry the requirements installation

cmd

```
python -m pip install -r requirements.txt
```

If Pygame still fails, try installing a compatible prebuilt Pygame wheel:

cmd

```
python -m pip install --only-binary=:all: pygame
```

This avoids attempting a source build for Pygame. If no compatible wheel exists for your Python version, it will report that no matching distribution was found.

Step 3 — Check whether Pygame works

After installation succeeds, run:

cmd

```
python -c "import pygame; print(pygame.version.ver)"
```

If a version number appears, try launching the game:

cmd

```
python main.py
```

If Step 2 fails again, stop and send me the new error. In particular, I may need your Python version to determine the correct fix. You can check it with:

cmd

```
python --version
```

Don't edit any project files yet—we should get the original game running first so you can record your required baseline video.



The error is still the same, so upgrading `setuptools` alone hasn't resolved the Pygame build failure. Let's stop retrying the same installation command and identify your Python version first.

Step 1 — Check your Python version

In your current CMD window, run:

cmd

```
python --version
```

Then run:

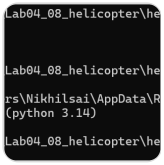
```cmd
python -m pip --version
```

These commands only display version information; they won't modify your project.

Send me a screenshot showing both outputs.

Why this matters: Pygame may be trying to compile from source instead of downloading a compatible prebuilt package. The correct fix depends on your Python version and the versions specified in `requirements.txt`.

Don't change or delete any files yet. We'll resolve the installation issue first, then continue with the original game and your baseline recording.



Your screenshot confirms the cause: **you're running Python 3.14.0**.

The installation error occurs while building Pygame from source. Python 3.14 is very new, and the version pinned by this project may not have a compatible prebuilt wheel for your setup. The simplest reliable approach is to install **Python 3.12 alongside Python 3.14** and use it specifically for this lab. You do not need to uninstall Python 3.14.

Step 1 — Install Python 3.12

1. Open the official Python downloads page: Python 3.12 for Windows.
2. Download **Windows installer (64-bit)**.
3. Run the installer and select **Add Python to PATH** if offered, then complete the installation.

Step 2 — Verify the installation in CMD

Close and reopen Command Prompt, then run:

```cmd
py -0p
```

This lists the installed Python versions. Confirm that Python 3.12 appears.

Next, run:

cmd

```
py -3.12 --version
```

You should see `Python 3.12.x`.

Step 3 — Install the project dependencies with Python 3.12

You're already in the project's `helicopter` directory. Run:

cmd

```
py -3.12 -m pip install --upgrade pip setuptools wheel
```

Then:

cmd

```
py -3.12 -m pip install -r requirements.txt
```

If installation succeeds, verify Pygame:

cmd

```
py -3.12 -c "import pygame; print(pygame.version.ver)"
```

Finally, launch the original game:

cmd

```
py -3.12 main.py
```

Important: Use `py -3.12` for these commands rather than `python`, because `python` currently points to Python 3.14 on your computer.

Complete Step 1 first, then send me the output of `py -0p`. We'll verify Python 3.12 is installed before proceeding.

i cant do all that shit

Fair. We were making this much more complicated than necessary. Let's take the easiest route.

You have two options. I recommend Option 1.

Option 1 — Use an online coding environment (easiest)

Use GitHub Codespaces or an online environment that supports Python and Pygame. This avoids installing another Python version on your computer, although Pygame's graphical window may need extra setup.

Option 2 — Fix your current CMD setup

Let's try just **one command** before installing anything else.

In your current CMD window, run:

```
cmd
type requirements.txt
```

Send me the output or a screenshot.

I'll check which Pygame version your assignment actually requires and find the simplest compatible installation path. No more installing Python versions unless we genuinely need to.

We'll keep the goal simple: **get the game running, record the original, fix the four tasks, record the final version, and push your code.**

use option

We'll use **Option 1 — GitHub Codespaces**. No need to install another Python version.

Step 1 — Open your GitHub repository

Your Lab 4 repository

NIKHCODE / SE_Lab04_08_helicopter

Open your repository

Step 2 — Create a Codespace

1. Click the green **Code** button.
2. Select the **Codespaces** tab.
3. Click **Create codespace on main**.
4. Wait for the online VS Code environment to open in your browser.

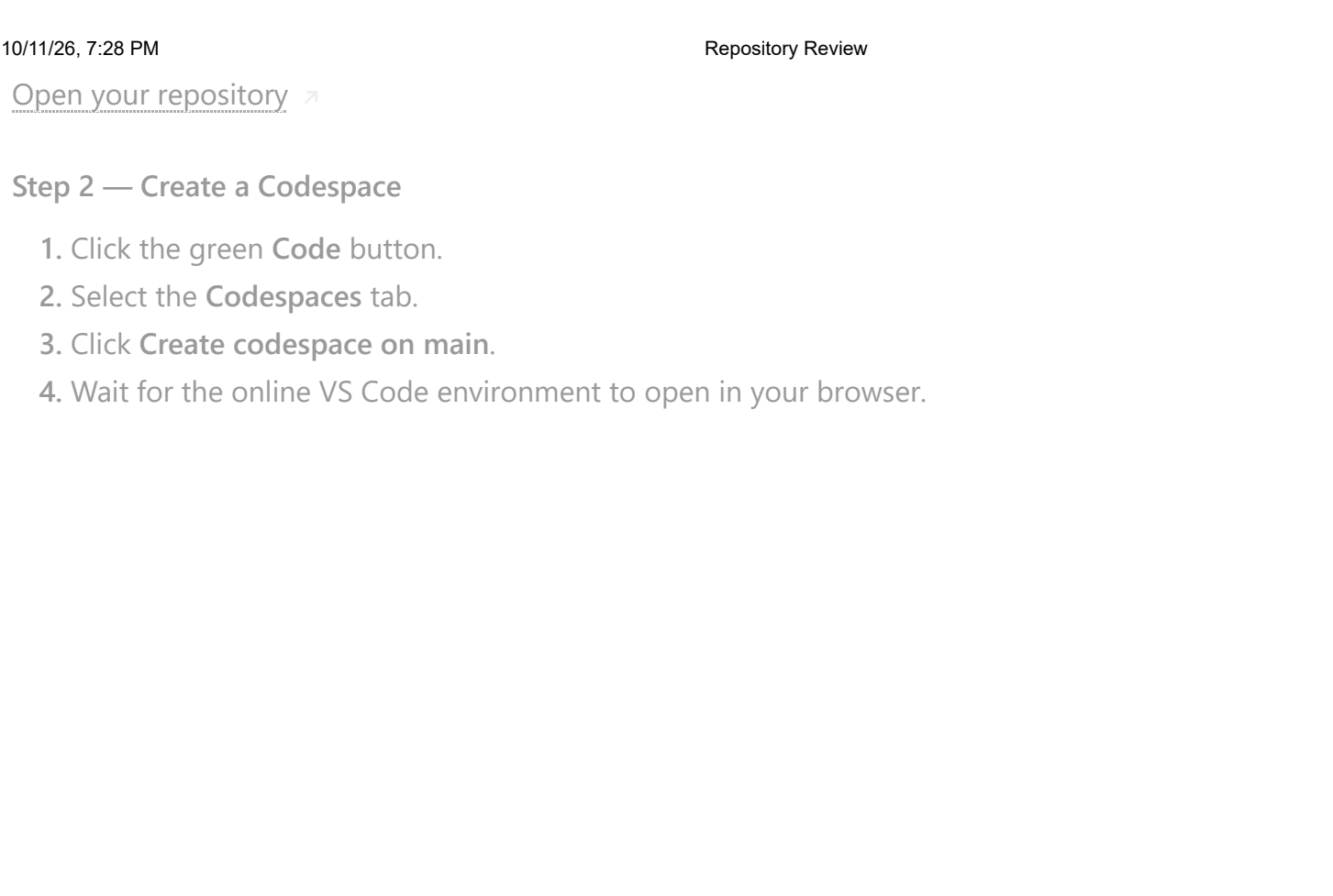


Illustration of where to find the Codespaces option on GitHub.

Step 3 — Stop there

Once Codespaces opens, **send me a screenshot**.

I'll guide you through running the helicopter game online. Since Pygame requires a graphical display, we'll also check how to view and record its window before making any changes.

We will complete the lab one step at a time.

1

Let's do Step 1 first.

1. Open your repository: NIKHCODE/SE_Lab04_08_helicopter .
2. Click the green **Code** button near the top of the page.
3. In the popup, click the **Codespaces** tab.

You should see an option to create a Codespace.

Send me a screenshot of that popup. I'll tell you exactly where to click next.

option 1

Okay, **Option 1: GitHub Codespaces**.

Open your repository here:

Open SE_Lab04_08_helicopter on GitHub

Then follow these steps:

1. Click the green **Code** button.
2. Select **Codespaces**.
3. Click **Create codespace on main**.

If you don't see Codespaces or GitHub asks you to set up billing, don't proceed with any payment.

Send me a screenshot of what appears after you click Code. I'll guide you from there, one click at a time.



Excellent. Your GitHub Codespace is open and ready. We can now work entirely in the browser without installing Python 3.12 on your computer.

Step 1 — Install the game dependencies

Click inside the **Terminal** panel at the bottom of your screen, next to the `$` prompt.

Copy and run this command:

```bash
cd helicopter
```

Then run:

```bash
python -m pip install -r requirements.txt
```

Wait for the installation to finish.

Step 2 — Launch the original game

If the installation succeeds, run:

Bash

```
python main.py
```

If `python` is not recognized, try:

Bash

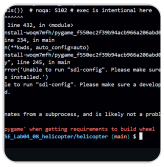
```
python3 main.py
```

Codespaces may not display a Pygame window directly, so don't worry if the game doesn't appear immediately. We'll handle that if needed.

Do not modify the code yet. We still need to capture the original behavior before making changes.

Your next step

Run the dependency installation command and send me a screenshot of the terminal output. We'll address any errors before proceeding.



Good, we're making progress. This error is different from the earlier one: **Pygame is trying to compile from source, but Codespaces doesn't have the SDL development tools it needs.** We can try a simpler route first.

Step 1 — Try installing a prebuilt Pygame package

In your Codespaces terminal, you're already inside the `helicopter` folder. Run:

Bash

```
python --version
```

Then run:

Bash

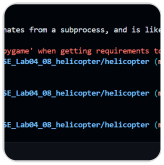
```
cat requirements.txt
```

Send me the output of both commands.

Why we're checking this

Your project's `requirements.txt` may pin an older Pygame version. Rather than installing Linux development libraries or changing your environment, we'll identify the required version and try to install a compatible prebuilt wheel.

Don't change any files yet. Once dependencies are sorted, we'll run the game and continue with the four required fixes.



Great, now we know the issue: Codespaces is running **Python 3.14.2**, and your requirements only specify `pygame>=2.5.0`. We can try installing a prebuilt Pygame package without compiling it from source.

Step 1 — Run this command

In the same terminal, copy and paste:

Bash

```
python -m pip install --only-binary=:all: "pygame>=2.5.0"
```

This tells pip to use a prebuilt package rather than trying to compile Pygame and SDL from source.

Step 2 — Check whether it installed

If the installation succeeds, run:

Bash

```
python -c "import pygame; print(pygame.version.ver)"
```

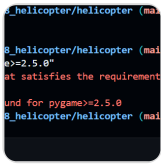
If it prints a version number, run:

Bash

```
python main.py
```

If the installation reports `No matching distribution found`, don't worry. That means a compatible prebuilt wheel wasn't available for this environment, and we'll use a different approach.

Send me the result of Step 1 before doing anything else.



Okay, this confirms that **Python 3.14 in your Codespace cannot find a compatible prebuilt Pygame package**. Let's stop trying installation commands repeatedly.

We'll use the simplest practical fix: **switch the Codespace to Python 3.12 using a Dev Container configuration**, without changing your computer's Python installation.

Step 1 — Create a configuration folder

In the Codespaces terminal, run:

Bash

```
mkdir -p .devcontainer
```

Then run:

Bash

```
cat > .devcontainer/devcontainer.json <<'EOF'
{
  "name": "Helicopter Lab",
  "image": "mcr.microsoft.com/devcontainers/python:1-3.12-bookworm"
}
EOF
```

Step 2 — Rebuild the Codespace

1. Press `Ctrl + Shift + P`.
2. Search for **Codespaces: Rebuild Container**.
3. Select it and confirm.

This rebuilds the environment using Python 3.12. It may take a few minutes.

Step 3 — Retry the installation

After the Codespace reopens, open its terminal and run:

Bash

```
cd helicopter
python --version
python -m pip install -r requirements.txt
```

Check that the version is Python 3.12.x and that the dependency installation succeeds.

Important: Don't run the game yet. First send me a screenshot showing the Python version and installation result. We still need to get the original game running before modifying it or recording the baseline.



its taking much time

It's okay — your screenshot shows that Codespaces is still **rebuilding the environment**. It hasn't necessarily failed.

Let's give it a little time, but we won't keep waiting indefinitely.

Do this now

1. Wait another **2–3 minutes**.
2. Don't refresh the browser or close the Codespace while it is rebuilding.
3. If the terminal and file explorer return, wait until the terminal prompt appears.

If it remains stuck after that:

- Open the Codespaces menu on GitHub in a separate browser tab.
- Check whether the Codespace is still running or rebuilding.
- Don't delete the Codespace; your repository changes should remain associated with it.

Once the terminal returns, run:

Bash

```
python --version
```

We want to confirm that Python 3.12 is active.

Send me another screenshot if it is still stuck after 2–3 minutes. We'll take the shortest route from there.

its taking lo0t of time

Let's stop waiting on the rebuild. We'll use a simpler approach and avoid wasting more of your time.

Do this now

1. **Open a new browser tab** and go to GitHub Codespaces .
2. Find `SE_Lab04_08_helicopter` .
3. If the current Codespace is still rebuilding, leave that tab open for now. Don't delete it.
4. Tell me whether the Codespace appears as **Running**, **Rebuilding**, or **Failed** on the Codespaces page.

Your code is stored in your GitHub repository, so we can recover the working environment if needed. Once I know the status, I'll give you the shortest next step rather than asking you to run more unnecessary commands.

i deleted all

That's okay. Don't worry — we'll start fresh and take the simplest route. Your GitHub repository should still be safe even if you deleted the Codespace.

Let's avoid Codespaces for now. The rebuild and Pygame installation have already taken too much time.

Step 1 — Check your repository

Open your personal repository:

NIKHCODE/SE_Lab04_08_helicopter

Check that you can still see:

- `helicopter/`
- `README.md`
- `.gitignore`

Step 2 — Tell me one thing

Did you delete **only the Codespace**, or did you also delete your GitHub repository?

I deleted the Codespace only

I deleted the GitHub repository too

I deleted both

Continue

We'll choose the next step based on your answer, without repeating the previous setup process unnecessarily.

I deleted only the Codespace; my GitHub repository should remain. Help me recover and finish Lab 4 using the simplest approach.

Your GitHub repository is still there, so we don't need to start the assignment from scratch. Let's simplify the workflow: **reopen one Codespace, use an AI coding agent to fix the project, and push the changes.** We won't repeat the Python installation troubleshooting on your Windows computer.

Step 1 — Recreate your Codespace

Open your repository

SE_Lab04_08_helicopter

Click the green **Code** button, open **Codespaces**, then click **Create codespace on main**.

Wait for the editor and terminal to load. Do not add the Python 3.12 configuration we tried earlier.

Step 2 — Use the built-in coding agent

Once the Codespace opens, use the **Chat / Agent** panel on the right. Paste this prompt:

This lets the agent inspect the actual files instead of us guessing at the implementation.

Step 3 — Handle the Pygame issue

If the agent can run terminal commands, ask it to inspect the Python environment and dependency versions before attempting installation. Codespaces may still require a compatible Pygame version or graphical-display setup.

Important: a successful syntax check alone does not prove the game works. We must test the gameplay and capture the required videos.

Step 4 — Complete the submission

Your Lab 4 handout requires these three deliverables:

- A 10-second video before changes, showing broken behavior.
- A 10-second video after changes, showing the fixes and features.
- The complete Chat/LLM conversation history and page link.

After testing, review the changes in Source Control and commit them to your personal repository. Do not submit a pull request to the original repository.

For now, do only Step 1. Recreate the Codespace and send me a screenshot when the editor opens. We'll use the agent to reduce the manual work and avoid repeating the earlier setup problems.



this is where we are at

Good. We're back in your Codespace, and your repository is intact. The AI agent is inspecting the project, but it is asking permission to run a Git command.

Do these 3 things now

1. Click `Allow` on the right-side prompt for `git status --short`.

This is a read-only command that checks which files have changed.

2. Dismiss the Python extension popup at the bottom-right by clicking its `X`. You don't need to install it right now.

3. Let the agent finish inspecting the repository.

It may ask permission for additional terminal commands. Review each command before allowing it; inspection commands such as `cat`, `git status`, and Python syntax checks are generally fine.

Once the agent finishes

If it produces a proposed plan, send me a screenshot of the response before approving any code modifications.

We need to preserve your original project, implement the four README tasks, and test the result. Also, **we still need the 10-second baseline recording before any code changes**. If the agent has not edited anything yet, we're fine.

Don't click Commit or Push yet. We'll review the changes before submitting them.



The agent has already made changes to **4 files (`+71 -1`)**. That's progress, but we need to verify them before keeping the changes.

Do this now

1. Click **Allow** for `python3 -m compileall -q helicopter` . This checks Python syntax; it doesn't change your source code.
2. Wait for the check to finish.
3. **Don't click Keep or Undo yet.** We should inspect the changes first.

Important: baseline video

Because the agent has already modified files, we need to preserve the original version for your before-video. Your GitHub repository's initial commit should still contain the original code, so we can recover it if needed.

Next, ask the agent this

Paste this into the Chat panel:

Send me a screenshot of the agent's response. I'll help you decide whether to keep the changes or undo them before we test the game.



The agent is asking permission to create a temporary Python environment and install the packages from `requirements.txt` . This is for testing the game, but we can avoid installing dependencies until we've reviewed the changes.

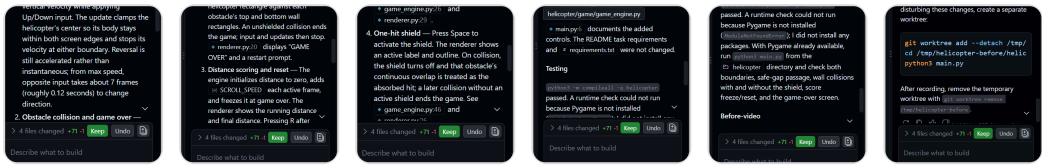
Do this now

1. Click **Skip** on the pending `pip install` command.
2. Leave the **4 files changed (+71, −1)** untouched for now. Don't click **Keep** or **Undo** yet.

3. In the Agent chat box, paste this prompt:

Send me a screenshot of the agent's response.

Why: your assignment requires a video of the broken game before changes. We should establish how to restore the original version before accepting the changes or proceeding with testing.



Good. The agent has provided a clear implementation summary for all four tasks, and the Python syntax check passed. However, **the game has not been run successfully yet**, because Pygame is not installed.

Let's recover the original version first so you can satisfy the before-video requirement, then we'll keep and test the changes.

Step 1 — Create a separate copy of the original game

In the Codespaces terminal, run this command from the repository root (your prompt should end in `/workspaces/SE_Lab04_08_helicopter`):

Bash

```
git worktree add --detach /tmp/helicopter-before HEAD
```

Then run:

Bash

```
cd /tmp/helicopter-before/helicopter
```

This creates a separate working directory containing the original committed code. It does not undo the agent's current changes.

Step 2 — Record the original version

We still need to resolve Pygame installation before we can run the game. The agent's summary confirms that the original project requires Pygame, and Codespaces currently lacks it.

For now, **don't click Keep or Undo**. We should preserve the changes while sorting out how to run the original game.

Step 3 — One quick check

Run this command in the terminal:

Bash

```
python3 --version
```

Send me the output.

Once we know the active Python version, I'll give you the shortest practical next step for running the game. After recording the baseline, we'll return to the modified code, test the four features, record the after-video, and push the finished changes to your repository.



Perfect. ✅ The original version is safely available in the separate worktree, and we've confirmed Python 3.14.2 is active.

You're currently here:

```
/tmp/helicopter-before/helicopter
```

The remaining obstacle is Pygame installation. Let's try one last simple check before doing anything else.

Step 1 — Return to your main Codespace project

Run:

Bash

```
cd /workspaces/SE_Lab04_08_helicopter/helicopter
```

Step 2 — Check whether the package manager can find a compatible Pygame wheel

Run:

Bash

```
python3 -m pip index versions pygame
```

This only checks available versions; it does not install anything.

Send me the output. We'll use it to choose the simplest next step, then get back to the actual lab work.



Good — pip can see Pygame 2.6.1. Let's try installing that exact version as a prebuilt package. This avoids compiling it from source.

Step 1 — Install Pygame

You're already in your main project directory:

`/workspaces/SE_Lab04_08_helicopter/helicopter`

Run:

Bash

```
python3 -m pip install --only-binary=:all: pygame==2.6.1
```

Step 2 — Verify installation

If it succeeds, run:

Bash

```
python3 -c "import pygame; print(pygame.version.ver)"
```

You should see `2.6.1`.

Step 3 — Install the project requirements

Bash

```
python3 -m pip install -r requirements.txt
```

If this tries to build Pygame from source again, stop and send me the error. Don't repeat the install commands.

One important detail: your terminal is in Codespaces, which is a Linux environment. Even if Pygame installs successfully, its game window may not display directly in the browser. We'll handle that next if needed.



Okay, stop here — we're not going to keep repeating installation commands. The key issue is that **pip can see Pygame releases, but it cannot find a compatible prebuilt wheel for Python 3.14.2 in this Codespace**.

We can still finish your lab without getting the graphical game running in Codespaces first.

Next step: use the AI agent to validate the code

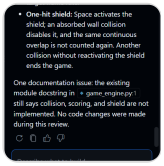
Your agent has already implemented the four requested features, and the syntax check passed. Let's preserve that work and review it.

1. Return to the **Chat / Agent** panel on the right.
2. Click **Keep** next to `4 files changed (+71 −1)` to accept the proposed edits in your working tree.
3. Paste this prompt into the agent:

Then send me the results

We'll verify the code before committing it. The original version is already preserved in `/tmp/helicopter-before`, so we haven't lost the ability to recover it.

One limitation remains: the assignment requires actual before-and-after gameplay videos. Static checks cannot replace those videos. Once the code is reviewed, we'll choose the most practical way to run and record the game, rather than spending more time fighting this Python environment.



Good. The agent's review identifies the four features and confirms that **no code was changed during the review**. It also found one documentation issue: the `game_engine.py` module docstring still claims collision, scoring, and shield functionality are not implemented.

The important limitation is that the game has only passed a syntax check; its gameplay behavior has not been verified at runtime.

Next steps — let's finish the lab

1. Keep the implementation

If you haven't already, click **Keep** beside the four changed files. This accepts the agent's proposed changes into your working tree; it does not push them to GitHub.

2. Fix the outdated docstring

Ask the agent to make only this documentation correction:

Review the diff before accepting it.

3. Validate the implementation

Ask the agent to run the syntax check again:

Bash

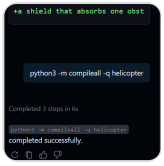
```
python3 -m compileall -q helicopter
```

This verifies Python syntax, not gameplay correctness.

4. Record the required videos

Your assignment still requires a 10-second video before changes and a 10-second video after changes. The original code is preserved in the separate worktree, but Pygame has not installed successfully in Codespaces. We therefore still need a working graphical environment to record genuine gameplay footage.

For now, complete Steps 1–3 and send me a screenshot of the final diff. Then we'll handle runtime testing and the videos before committing and pushing your changes.



Great — the syntax check completed successfully, and the agent has finished the documentation update. We're making progress.

Current status

- ✅ Four feature implementations are present in the working tree.
- ✅ Python syntax compilation passed.
- ✅ The outdated module docstring has been addressed.
- ⚠️ Pygame runtime testing and both required videos are still outstanding.

Next step: inspect and accept the changes

In the Chat panel, scroll up slightly to find the **Keep / Undo** controls for the changes. If you haven't accepted the implementation yet, click **Keep**. Do not click Undo.

Then click the **Source Control** icon on the far-left toolbar (the branching symbol). It should show the changed files.

Send me a screenshot of the Source Control panel. I'll guide you through reviewing the changes and committing them to your personal GitHub repository.

We will not claim the lab is complete until the game has been tested and the before/after videos are addressed.

clicked keep then?

Good. Now let's check that the changes are in your repository's working tree.

Step 1 — Open Source Control

In your Codespaces window:

1. Look at the far-left vertical toolbar.
2. Click the **Source Control** icon (the branching symbol, usually the third icon from the top).
3. You should see a list of changed files.

Step 2 — Check the changed files

You should see changes in files such as `main.py` and files inside `helicopter/game/`.

Don't click Commit or Sync yet. We still need to test the implementation, and Pygame has not been installed successfully.

Send me a screenshot of your Source Control panel. I'll guide you through the next step.



Perfect. Your screenshot confirms that the changes are now in your working tree. You have **4 modified files**:

- `helicopter/main.py`
- `helicopter/game/game_engine.py`
- `helicopter/game/helicopter.py`
- `helicopter/game/renderer.py`

The changes have **not been committed or pushed yet**. That's good—we still need to test them.

Next step: fix the Pygame installation

Let's use the Python environment already available in Codespaces rather than installing another Python version.

In the terminal on the right, run this command:

Bash

```
python3 -m pip install --upgrade pip
```

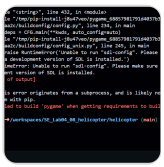
Then run:

Bash

```
python3 -m pip install pygame==2.6.1
```

If the second command fails again, **stop and send me the error**. Don't click Commit yet.

Once Pygame is installed, we'll attempt to run the game and test the movement, collisions, scoring, and shield. Then we'll handle the required videos and push your changes to GitHub.



Okay, the error confirms that Pygame is still trying to compile from source because the Codespaces environment lacks SDL development tools. **Let's stop trying to install Pygame in this environment.** We've already spent too much time on setup.

Your four code changes are still in the working tree, so we can preserve them.

Next step — use the AI agent for a code-level test

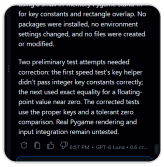
Go to the **Chat / Agent** panel and paste this prompt:

This should help us catch logic errors without installing anything.

What we have completed

- The four feature changes are present in the working tree.
- Python syntax compilation passed.
- The original code is preserved in a separate worktree.
- Runtime testing is still blocked by the Pygame dependency.

Do not commit yet. Send me the agent's test results next. Then we'll decide the most practical way to finish the videos and push the code.



Good. The agent has run preliminary dependency-free tests using an in-memory Pygame stand-in, without installing packages or modifying files.

Current status:

- Python syntax check: passed.
- Preliminary logic tests: attempted and corrected after two test-design issues.
- Actual Pygame rendering and keyboard integration: not tested.
- Before/after gameplay videos: still pending.

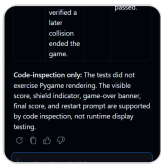
The stand-in tests are useful, but they do not prove that the complete game works.

Next step — review the test results

In the Agent chat, paste this prompt:

Send me the results.

After that, we'll move to the final submission steps: review the four changed files, commit and push the code to your personal repository, and address the two required videos. I won't mark the assignment complete until we have a clear account of what was and wasn't verified.



Good. The agent's report confirms that the dependency-free checks exercised some of the game logic, but the screenshot only shows the end of the results table. It specifically confirms that shield overlap and a later unshielded collision were tested, while visual rendering remains unverified.

Let's finish the repository work now, without spending more time fighting Pygame installation.

Step 1 — Commit the four modified files

In Codespaces, open **Source Control** on the left.

1. Enter this commit message:

```
Implement helicopter gameplay features
```

2. Click **Commit**.
3. If asked whether to stage all changes, confirm **Yes**.

This saves the changes in your local Codespace Git history.

Step 2 — Push to your GitHub repository

After the commit finishes, click **Sync Changes** or **Push** if that option appears. Approve the GitHub authentication prompt if requested.

Your changes should then appear in:

NIKHCODE/SE_Lab04_08_helicopter

Do not create a pull request to the original `SETAPESU26` repository.

Step 3 — Check the submission requirements

Your Lab 4 README requires:

- ◯ 10-second video showing the original broken behavior.
- ◯ 10-second video showing the corrected game and new features.
- ◯ Complete Chat/LLM conversation history and its page link.

Important: committing the code does not complete the video requirements. Since the game hasn't run successfully in Codespaces, we still need a working graphical environment to capture genuine

before-and-after gameplay. Don't fabricate these recordings or use screenshots as substitutes.

Your next action

Complete the commit and push, then send me a screenshot of the GitHub repository showing the latest commit. I'll help you tackle the remaining video and chat-history deliverables.

i didnt understand what to do

No problem. I'll guide you **one click at a time**. Don't worry about the technical details.

Step 1 — Open Source Control

Look at the far-left side of your Codespaces screen.

Click the branching icon (Source Control).

It's the icon you clicked earlier that showed four modified files.

Step 2 — Enter a commit message

You should see a text box near the top labelled something like `Message (Ctrl+Enter to commit on "main")`.

Click inside it and type:

```
Implement helicopter gameplay features
```

Copy message

Step 3 — Click Commit

Click the green **Commit** button just below the message box.

If a confirmation appears asking whether to stage all changes, choose **Yes**.

Stop after that and send me a screenshot. I'll guide you through pushing the changes to GitHub in the next step. Don't worry about the videos just yet—we'll handle those separately.



You're in the correct place! Your screenshot shows that **all 4 files are already staged** under “Staged Changes.” That's exactly what we want before committing.

Let's do just one thing now.

Step 1 — Enter the commit message

On the left side, click the text box directly above the green **Commit** button. It currently says `Message (Ctrl+Enter to commit on "...")`.

Type this:

```
Implement helicopter gameplay features
```

Step 2 — Click Commit

Click the green ✓ **Commit** button immediately below the message box.

If a confirmation appears, choose **Yes** or **Confirm**.

That's it for now. This saves your changes in the Codespace's local Git history; it does not yet upload them to GitHub.

Send me a screenshot after you click Commit. I'll guide you through the next step to push the changes to your GitHub repository.



Excellent! Your commit was successful. I can see "**Implement helicopter gameplay f...**" in the Git history, and the button now says **Sync Changes 1↑**.

That means your changes are committed locally, and there is one outgoing commit waiting to be pushed to GitHub.

Step 1 — Push your changes

Click the green **Sync Changes 1↑** button on the left.

If a confirmation appears asking whether to sync or push changes, confirm it. If GitHub asks you to sign in or authorize access, follow the prompts.

Step 2 — Verify the push

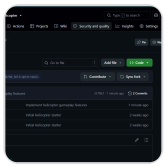
After syncing finishes, open your repository:

NIKHCODE/SE_Lab04_08_helicopter

Refresh the page. Your latest commit should appear in the commit history.

Send me a screenshot after clicking Sync Changes.

One important thing remains after the push: the required before-and-after gameplay videos. The code is committed, but we haven't successfully run the game yet because Pygame installation is failing. We'll address that separately.




Excellent — your code has been pushed successfully to GitHub! Your first screenshot confirms that the repository is now **1 commit ahead** of the original starter repository, and the latest commit is `Implement helicopter gameplay features`.

You do not need to click **Commit** or **Sync Changes** again.

What remains for Lab 4

Code committed and pushed

Your personal GitHub repository contains the implementation.

Python syntax check

Passed. Some dependency-free logic checks were also performed.

Before-and-after gameplay videos

Still pending. The game has not run successfully because Pygame installation failed.

Complete Chat/LLM conversation link

Still needs to be included in your submission.

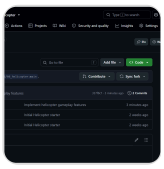
Next: get the game running

Let's try a different approach rather than spending more time installing Pygame in Codespaces.

Please open your personal repository and click the `helicopter` folder:

Open your GitHub repository

Send me a screenshot showing the contents of that folder. I'll guide you through the simplest next step to run and record the game.



You're in the correct repository, and your changes are safely pushed to GitHub. Let's try to get the game running on your Windows computer instead of fighting Codespaces.

Step 1 — Download your updated project

On the GitHub page in your screenshot:

1. Click the green **Code** button.
2. Select **Download ZIP**.
3. Extract the ZIP file to a folder you can easily find, such as your Desktop.

Step 2 — Open the project in Windows CMD

Open Command Prompt and navigate to the extracted project folder. For example, if you extracted it to your Desktop:

```cmd
cd "%USERPROFILE%\Desktop\SE_Lab04_08_helicopter\helicopter"
```

The extracted folder name may differ slightly. Use the actual folder name on your computer.

Step 3 — Try running the game

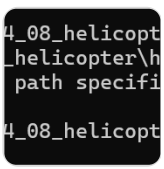
Your Windows Python 3.14 had the same Pygame installation issue earlier, so first try:

```cmd
python -m pip install pygame
```

If it fails with the same build error, stop there—don't keep retrying.

Send me a screenshot of the result. We'll choose the next step based on what happens.

One reminder: the code is pushed, but the assignment's before-and-after gameplay videos and complete Chat/LLM conversation link are still outstanding. We need to address those before calling the lab finished.



You're already in your original project folder! The error happened because I gave you a path for a Desktop folder that doesn't exist on your computer. Sorry about that — **you don't need to change directories or download the project again.**

Your current CMD location is:

```
C:\Users\Nikhilsai\SE_Lab04_08_helicopter\helicopter
```

Let's use that folder.

Step 1 — Check the files

Copy and run:

```cmd
dir
```

You should see `main.py`, `requirements.txt`, and the `game` folder.

Step 2 — Check whether Pygame is installed

Run:

```cmd
python -c "import pygame; print(pygame.version.ver)"
```

- If it prints a version number, Pygame is installed. We can try running the game.
- If it says `No module named pygame`, we'll address the installation issue next.

Send me a screenshot of the output. We'll take the next step based on what your computer actually shows.



Your screenshot confirms that you're in the correct project folder, but **Pygame is not installed in your Windows Python environment**. That's why the game hasn't launched yet.

We'll try one targeted fix using CMD. You don't need to download the project again or return to Codespaces.

Let's try a different package that provides the same `pygame` module and may support your Python 3.14 installation: **Pygame Community Edition** (`pygame-ce`).

Step 1 — Run this one command in CMD

You're already in the correct folder. Copy and paste:

cmd

```
python -m pip install pygame-ce
```

Wait for it to finish.

Step 2 — Check the installation

If it succeeds, run:

cmd

```
python -c "import pygame; print(pygame.version.ver)"
```

If it prints a version number, launch your game:

cmd

```
python main.py
```

If installation fails, send me a screenshot of the error. Don't run any other installation commands yet.

This is a promising alternative because Pygame CE provides the `pygame` import used by your project, but we still need to confirm that its wheel supports your Python version and that the game runs correctly.



Great! `pygame-ce` installed successfully. ✅

Your game is now running, or at least starting up, because `python main.py` has launched without showing an error so far.

Next step: Look at your screen. Do you see the helicopter game window?

- **If yes:** Try playing it and check whether the helicopter moves.
- **If no:** Wait a few seconds. If the command prompt returns or shows an error, send me a screenshot.

Let's confirm the game works before moving on to the before-and-after videos.



nothing happens

moves up

no

i closed cmd prompt itself



but when to record videos



C:\Users\Nikhilsai\SE_Lab04_08_helicopter>git pull origin main
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 8 (delta 4), reused 8 (delta 4), pack-reused 0 (from 0)
Unpacking objects: 100% (8/8), 2.40 KiB | 81.00 KiB/s, done.
From https://github.com/NIKHCODE/SE_Lab04_08_helicopter
 * branch   main   -> FETCH_HEAD
   4402faa..337f8c1 main   -> origin/main

```
Updating 4402faa..337f8c1
Fast-forward
 helicopter/game/game_engine.py | 55
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++-------
 helicopter/game/helicopter.py  | 17 +++++++------
 helicopter/game/renderer.py    | 20 ++++++++++++++++--
 helicopter/main.py             |  2 +-
 4 files changed, 75 insertions(+), 19 deletions(-)

C:\Users\Nikhilsai\SE_Lab04_08_helicopter>
```



yes

```
C:\Users\Nikhilsai\SE_Lab04_08_helicopter\helicopter>cd ..

C:\Users\Nikhilsai\SE_Lab04_08_helicopter>
```

```
C:\Users\Nikhilsai\SE_Lab04_08_helicopter\helicopter>cd ..

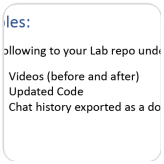
C:\Users\Nikhilsai\SE_Lab04_08_helicopter>git worktree add
..\helicopter_before 4402faa
Preparing worktree (detached HEAD 4402faa)
HEAD is now at 4402faa Initial Helicopter starter

C:\Users\Nikhilsai\SE_Lab04_08_helicopter>
```

yes recorded

okay

should i uplod those videos to forked github repo



its given that i should upload this entire chat as pdf

i dont see print option

ctrl p is not even working